St. Thomas Institute for Science & Technology
Kazhakoottam, Thiruvananthapuram– 695584

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PBCST304: OBJECT ORIENTED PROGRAMMING

MODULE 4

SYLLABUS

SOLID Principles in Java (https://www.javatpoint.com/solid-principles java)

Swings fundamentals – Overview of AWT, Swing v/s AWT, Swing Key Features, Model View Controller (MVC), Swing Controls, Components and Containers, Swing Packages, Event Handling in Swings, Swing Layout Managers, Exploring Swings–JFrame, JLabel, The Swing Buttons, JTextField.

Event handling – Event Handling Mechanisms, Delegation Event Model, Event Classes, Sources of Events, Event Listener Interfaces, Using the Delegation Event Model.

Developing Database Applications using JDBC – JDBC overview, Types, 10 Steps, Common JDBC Components, Connection Establishment, SQL Fundamentals [For projects only] - Creating and Executing basic SQL Queries, Working with Result Set, Performing CRUD Operations with JDBC.

SOLID Principles in Java

The SOLID principles are five important rules that help make software design better. They make the code easier to understand, change, and reuse. These rules are:

- Single Responsibility Principle (SRP) – A class should do only one job.
- Open/Closed Principle – Code should be open to adding new features but closed to changing existing code.
- Liskov Substitution Principle (LSP) – You should be able to use a child class wherever a parent class is used, without problems.

- Interface Segregation Principle (ISP) – Don’t force a class to use methods it doesn’t need.
- Dependency Inversion Principle (DIP) – Depend on abstractions (interfaces), not on concrete (real) classes.

The SOLID principle helps in reducing tight coupling. Tight coupling means a group of classes are highly dependent on one another which you should avoid in your code.

- Opposite of tight coupling is loose coupling and your code is considered as a good code when it has loosely-coupled classes.
- Loosely coupled classes minimize changes in your code, helps in making code more reusable, maintainable, flexible and stable. Now let's discuss one by one these principles...

1.S-Example

```
// WRONG: Both logic and printing in one class
class Employee {
    public void calculateSalary() { /*...*/ }
    public void printDetails() { /*...*/ }
}

// RIGHT: Split responsibilities
class SalaryCalculator {
    public void calculateSalary() { /*...*/ }
}

class EmployeePrinter {
    public void printDetails() { /*...*/ }
}
```

O-Example

```
// WRONG: You have to modify the class for new shape
class AreaCalculator {
```

```
    public double calculateArea(Object shape) {
        if (shape instanceof Circle) { /*...*/ }
        else if (shape instanceof Rectangle) { /*...*/ }
        return 0;
    }
}

// RIGHT: Use inheritance and interfaces
interface Shape {
    double area();
}

class Circle implements Shape {
    public double area() {
        return 3.14 * 5 * 5;
    }
}

class Rectangle implements Shape {
    public double area() {
        return 10 * 5;
    }
}

class AreaCalculator {
    public double calculateArea(Shape shape) {
        return shape.area();
    }
}
```

L-Example

```
class Bird {
    public void fly() {
        System.out.println("Bird is flying");
    }
}

class Sparrow extends Bird { }

class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Ostrich can't fly");
    }
}
// ❌ Violates LSP because Ostrich can't fly

// ☑ Better approach: Separate flying birds
class Bird { }

class FlyingBird extends Bird {
    public void fly() { }
}

class Sparrow extends FlyingBird { }
class Ostrich extends Bird { }
```

I-Example

```
// WRONG: One interface for all devices
interface Machine {
```

```
    void print();
    void scan();
    void fax();
}

// RIGHT: Split into smaller interfaces
interface Printer {
    void print();
}

interface Scanner {
    void scan();
}

class SimplePrinter implements Printer {
    public void print() { /*...*/ }
}
```

D-Example

```
        // WRONG: Tight coupling
        class MySQLDatabase {
            public void connect() { }
        }

        class App {
            MySQLDatabase db = new MySQLDatabase(); // depends on concrete class
        }

        // RIGHT: Use interface
        interface Database {
```

```
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() { }
}

class App {
    Database db;
    public App(Database db) {
        this.db = db;
    }
}
```

EVENT

- Change in the state of an object is known as event i.e. event describes the change in state of source.
- Events are generated as result of user interaction with the graphical user interface components.
- For example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, scrolling the page are the activities that causes an event to happen.

Types of Event

- The events can be broadly classified into two categories:

Foreground Events

- Those events which require the direct interaction of user.
- They are generated as consequences of a person interacting with the graphical components in Graphical User Interface. For example, clicking on a button, moving the mouse, entering a character through keyboard selecting an item from list, scrolling the page etc.

Background Events

- Those events that require the interaction of end user are known as background events.
- Operating system interrupts, hardware or software failure, timer expires, an operation completion are the example of background events.

EVENT HANDLING